Lab 47 — Supply Chain Security: SBOM and Vendor Trust

> **SY0-701 Objective 5.3** — Explain the processes associated with third-party risk assessment and management
>
> **Domain 5** — Security Program Management and Oversight (20%) · **Time:** ~20 min · **Course:** TGS-2023039181 CompTIA Security+ (SY0-701)

Goal: Produce a software bill of materials and verify that a vendor download is authentic before you trust it.

You'll build: An SBOM of installed software, a vendor download verified by SHA-256 checksum and GPG signature, and a vendor-trust checklist capturing right-to-audit and questionnaire items.

Prerequisites: Killercoda Ubuntu Playground (root shell) — https://killercoda.com/playgrounds/scenario/ubuntu ; `syft` and `gnupg` installed in step 1.

Background

- **Supply chain analysis** maps the software and vendors you depend on; their weaknesses become yours.
- A **software bill of materials (SBOM)** is an inventory of every component and version in a product — the ingredient list that lets you answer "are we affected?" when a CVE drops.
- Before trusting a vendor download, verify **integrity** (a checksum proves the file was not altered) and **authenticity** (a GPG signature proves it came from the vendor).
- Vendor trust is also contractual: a **right-to-audit clause**, security **questionnaires** and evidence of **independent assessments** (SOC 2, ISO 27001) are part of third-party risk management.

Steps

1. Install the tools

Syft is a single binary installed by the vendor's official script; also install GnuPG and curl.

```
apt-get update -qq && apt-get install -y curl gnupg coreutils
curl -sSfL https://raw.githubusercontent.com/anchore/syft/main/install.sh | sh -s -- -b
/usr/local/bin
syft version
```

2. Generate an SBOM of installed software

Syft catalogues packages into a standard SBOM format (SPDX).

```
mkdir -p ~/supplychain && cd ~/supplychain
syft dir:/usr/bin -o spdx-json=sbom.spdx.json
ls -l sbom.spdx.json
```

3. Read the SBOM component list

The table view shows each component and its version — the data you query against a CVE feed.

```
cd ~/supplychain
syft dir:/usr/bin -o table | head -20
```

```
NAME        VERSION     TYPE
bash        5.1-6       deb
coreutils   8.32-...    deb
...
```

4. Download a vendor release and its published hashes

We use a real, stable GitHub release (jq) that publishes checksums and signatures.

```
cd ~/supplychain
BASE=https://github.com/jqlang/jq/releases/download/jq-1.7.1
curl -sSLO $BASE/jq-linux-amd64
curl -sSLO $BASE/sha256sum.txt
ls -l jq-linux-amd64 sha256sum.txt
```

5. Verify the SHA-256 checksum (integrity)

A matching hash proves the bytes were not tampered with in transit.

```
cd ~/supplychain
grep 'jq-linux-amd64' sha256sum.txt > my-sum.txt
sha256sum -c my-sum.txt
```

```
jq-linux-amd64: OK
```

6. Verify the GPG signature (authenticity)

A signature verified against the vendor's public key proves the file's origin. (If no key is imported, verification fails loudly — which is the point.)

```
cd ~/supplychain
curl -sSLO https://github.com/jqlang/jq/releases/download/jq-1.7.1/jq-linux-amd64.asc
2>/dev/null || true
gpg --verify jq-linux-amd64.asc jq-linux-amd64 2>&1 | head -5 || \
  echo "Signature check ran — import the vendor key to get a trusted result."
```

7. Write the vendor-trust checklist

Technical verification plus contractual assurance together make up third-party due diligence.

```
cd ~/supplychain
cat > vendor-trust.md <<'EOF'
# Vendor Trust Checklist
Vendor: Example SaaS Pte Ltd
```

```
[ ] SBOM obtained and scanned against CVE feed
[x] Download integrity verified (SHA-256)
[ ] Download authenticity verified (GPG signature)
[ ] Right-to-audit clause present in the contract
[ ] Security questionnaire (SIG/CAIQ) returned and reviewed
[ ] Independent assessment on file (SOC 2 / ISO 27001)
[ ] Supply chain analysis: their sub-processors listed
EOF
cat vendor-trust.md
```

Test it

```
cd ~/supplychain
test -s sbom.spdx.json && echo "SBOM generated"
sha256sum -c my-sum.txt
grep -c '\[' vendor-trust.md
```

Expected: `sbom.spdx.json` is non-empty and lists components with versions; the checksum reports `jq-linux-amd64: OK`; and `vendor-trust.md` records right-to-audit, questionnaire and supply-chain items.

Troubleshooting

Symptom	Fix
`syft: command not found`	The install script failed (no network); re-run step 1, or install from the GitHub release page.
`sha256sum -c` says FAILED	The download was incomplete or altered; delete and re-download in step 4.
`gpg --verify` says "No public key"	Expected without the vendor key; import it with `gpg --recv-keys <KEYID>` to get a trusted result.

Challenge

Scan the SBOM against a vulnerability database with `grype` (same vendor as Syft) and report how many of the catalogued components have known CVEs.

Reflection

Objective 5.3 lists right-to-audit, questionnaires and supply chain analysis. Why is verifying a download's signature not enough on its own to trust a vendor?

Exam tips

- SBOM = the component/version inventory that lets you answer "are we exposed to this CVE?".
- Checksum proves **integrity** (unaltered); a digital signature proves **authenticity** (origin).
- Right-to-audit clause = your contractual permission to inspect a vendor's security.
- Independent assessments (SOC 2, ISO 27001) are third-party attestations; a vendor's internal audit is self-reported.

References

- NTIA minimum elements for an SBOM — https://www.ntia.gov/report/2021/minimum-elements-software-bill-materials-sbom
- Anchore Syft — https://github.com/anchore/syft
- CISA software supply chain security — https://www.cisa.gov/supply-chain
- SPDX SBOM standard — https://spdx.dev
